

Développement Web Services

Web Services

REST (JAX-RS)

1

Introduction au **services web**

Le protocole http

- HTTP : HyperText Transfer Protocol
- Protocole de communication développé pour le **Web**
- Protocole fonctionnant avec : **requête / réponse**
- Transfert de données entre un navigateur et un serveur Web
- **Sans état** (Stateless)

Le protocole http

Message de requête composé de :

- Une ligne de requête (**Request Line**) composée de:
 - La méthode de requête utilisée
 - L'URI de la ressource
 - Le protocole et la version utilisée
- En-têtes (**Headers**)
- Une ligne vide
- Un corps de message optionnel (**Message Body**)

Le protocole http

Message de requête

```
POST /en/html/index.html HTTP/1.1
Host: www.website.com
User-Agent: Mozilla/5.0 (Windows;en GB; rv:1.8.0.11)
Accept:
text/xml,text/html;q=0.9,text/plain;q=0.8,image/png,
*/*;q=0.5
Accept-Language: en-gb,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Content-Length: 39

name=MyName&male=yes
```

Le protocole http

Méthodes de requête

- **GET**: demande une représentation de la ressource
- **POST**: Soumettre les données à traiter à la ressource identifiée.
Cela peut entraîner la création d'une nouvelle ressource
- **PUT**: Télécharge une représentation de la ressource spécifiée
- **DELETE**: supprime la ressource spécifiée
- ...

Le protocole http

Codes d'état de la réponse

Les codes d'état sont divisés en cinq classes:

- **1xx: informatif**

Indique une réponse provisoire

- **2xx: succès**

Indique que la demande a été reçue, comprise, acceptée et traitée avec succès

Exemples :

200 OK

201 créées

Le protocole http

Codes d'état de la réponse

- **3xx: redirection**

Indique que l'agent utilisateur doit prendre d'autres mesures pour répondre à la demande.

Exemple :

301 Déménagé Définitivement

- **4xx: erreur client**

Destiné aux cas dans lesquels le client semble avoir commis une erreur

Exemples :

403 interdit

404 introuvable

405 Méthode non autorisée

Le protocole http

Codes d'état de la réponse

- **5xx: Erreur du serveur**

Indique que le serveur est conscient qu'il a rencontré une erreur ou est autrement incapable d'effectuer la demande.

Exemples :

500 Erreur interne du serveur

501 non mis en œuvre

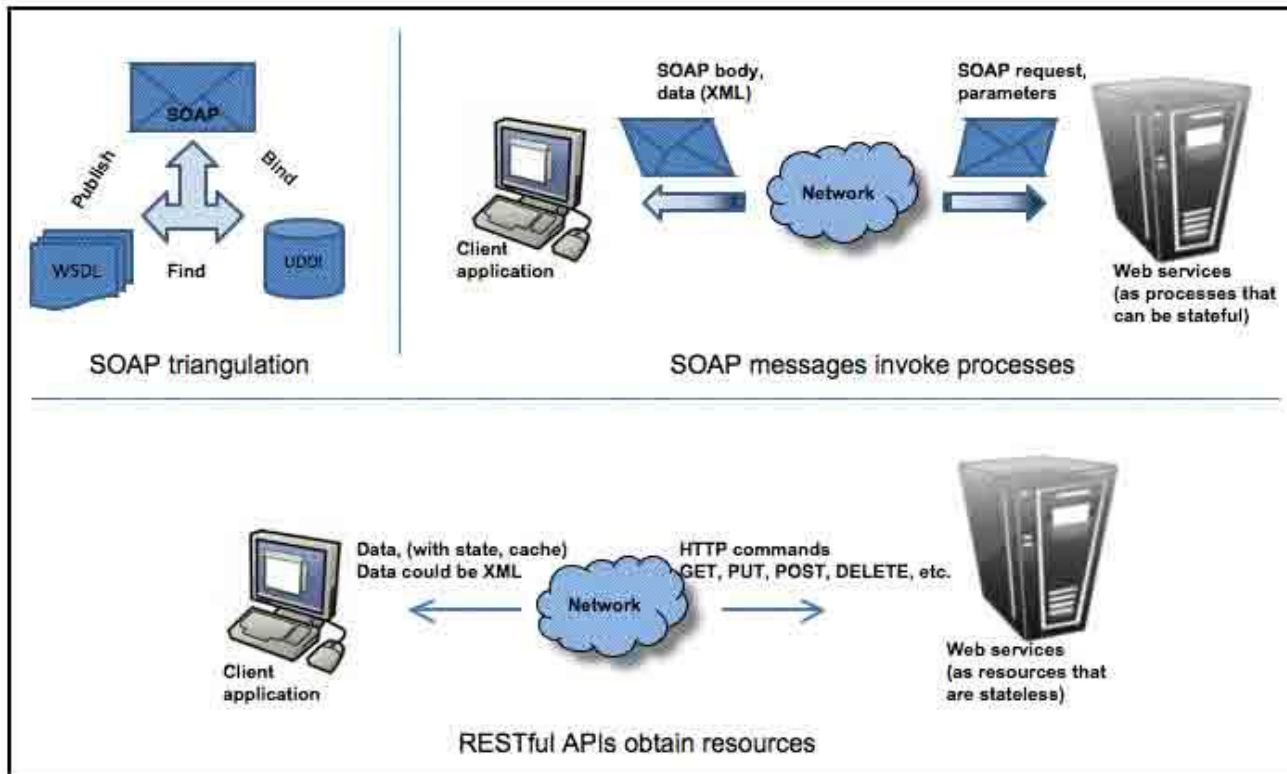
502 Mauvaise passerelle

Les services web

- Un service web est un système logiciel qui permet la communication entre deux ou plusieurs applications sur le web.
- **Objectifs :**
 - Partager la logique / données
(Sans fournir des interfaces graphiques)
- **Exemple:**
 - Les applications clientes Twitter utilisent les services Web Twitter pour récupérer / publier des tweets
- Les deux types de services Web les plus célèbres sont:
 - Services Web **REST** (également appelés : Web API)
 - Services Web **SOAP** (également appelés : Big Web Services)

Les Services Web

(SOAP vs REST)



2

Services web REST

Les services web REST

Le principe REST

Le terme vient de la thèse de doctorat de **Roy Fielding** :

REpresentational **S**tate **T**ransfer

Il définit un système REST avec les contraintes suivantes:

- doit être un système client-serveur
- doit être sans-état (stateless)
- doit supporter un système de cache
- doit être accessible uniformément
- doit être mis en couches (support évolutif)

Les services web REST

Les ressources

Une ressource REST est tout ce qui est adressable sur le Web.

Des exemples :

- La température à Rabat à 20h00
- Un article de blog
- Une liste de rapports de bugs
- Un résultat de recherche de Google
- etc.

Les services web REST

La représentation

Une représentation est généralement un document qui capture l'état actuel ou prévu d'une ressource.

C'est ce qui est envoyé entre les clients et les serveurs.

Peut prendre diverses formes telles que:

HTML Document

Plain Text

XML Stream

JSON Stream

etc.

Une ressource peut avoir plus d'une représentation

Les services web REST

Les actions

Les web services REST (également appelés RESTful Web APIs) : il s'agit d'un simple service web implémenté avec HTTP et les principes de REST.

L'URI d'une ressource est un lien hypertexte

Le protocole HTTP fournit des méthodes sur lesquelles on peut mapper les opérations CRUD (et appliquez différentes actions avec le même URI)

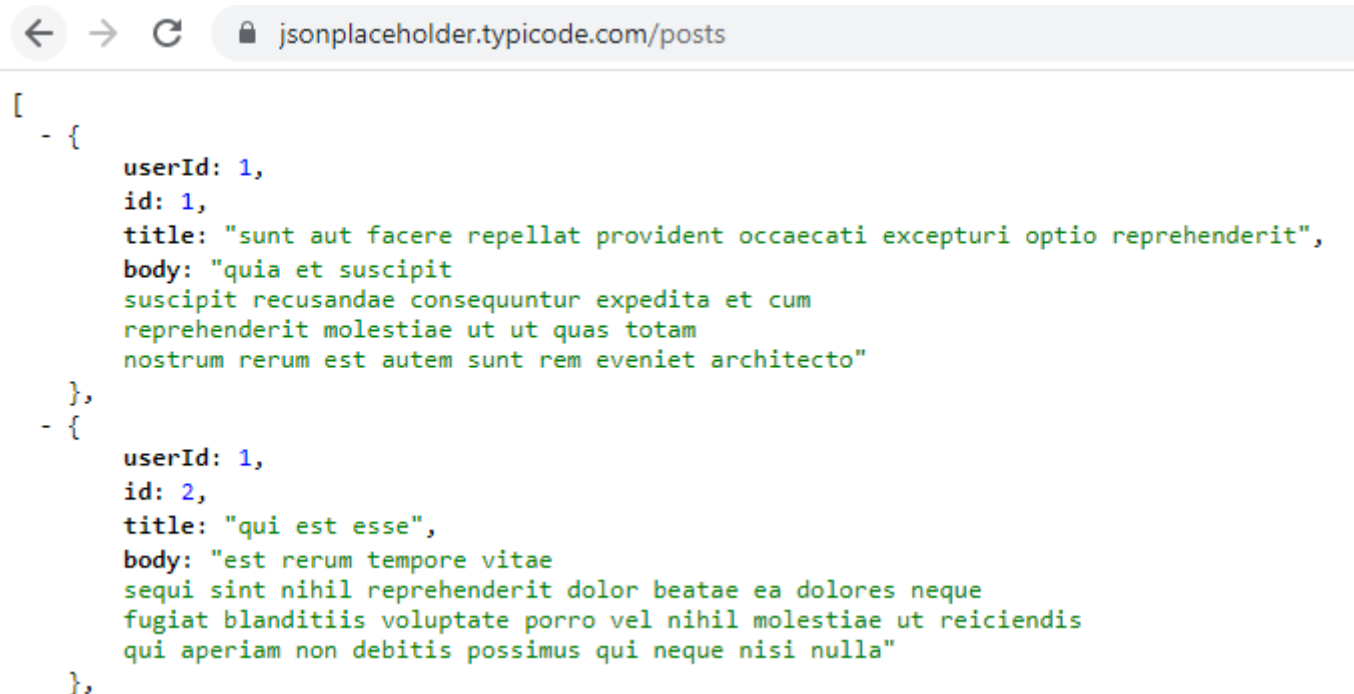
Data action	HTTP protocol equivalent
CREATE	POST
RETRIEVE	GET
UPDATE	PUT
DELETE	DELETE

Les services web REST

Un exemple

Des exemples de services Web REST sont disponibles à l'URL suivante:

<https://jsonplaceholder.typicode.com/>



```
[
  - {
    userId: 1,
    id: 1,
    title: "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    body: "quia et suscipit
    suscipit recusandae consequuntur expedita et cum
    reprehenderit molestiae ut ut quas totam
    nostrum rerum est autem sunt rem eveniet architecto"
  },
  - {
    userId: 1,
    id: 2,
    title: "qui est esse",
    body: "est rerum tempore vitae
    sequi sint nihil reprehenderit dolor beatae ea dolores neque
    fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis
    qui aperiam non debitis possimus qui neque nisi nulla"
  },
]
```

GET / Retrieve

La méthode GET est utilisée pour récupérer des ressources

Par exemple, une demande GET à **URI/students** renvoie tous les étudiants:

```
<students>
  <count>2</count>
  <student>
    <birthDate>5287-09-15T22:00:13.524Z</birthDate>
    <firstName>Jack</firstName>
    <idBooster>59253</idBooster>
    <lastName>Harkness</lastName>
  </student>
  <student>
    <birthDate>1997-08-12T22:00:13.524Z</birthDate>
    <firstName>Eric</firstName>
    <idBooster>64543</idBooster>
    <lastName>Cartman</lastName>
  </student>
</students>
```

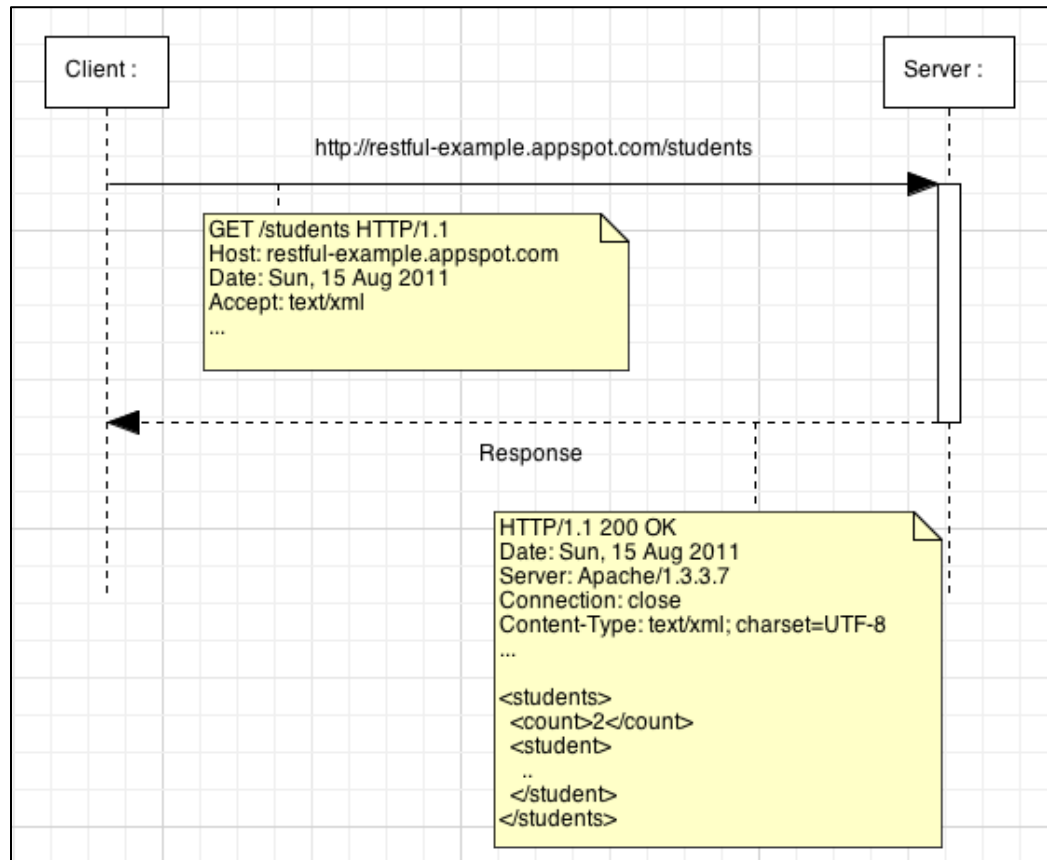
GET / Retrieve

Une demande GET à **URI/students/59253** renvoie l'étudiant **Jack**

```
<student>
  <birthDate>5287-09-15T22:00:13.524Z</birthDate>
  <firstName>Jack</firstName>
  <idBooster>59253</idBooster>
  <lastName>Harkness</lastName>
</student>
```

GET / Retrieve

Les détails de la requête



POST/ Retrieve

La méthode POST est utilisée pour CREER des ressources

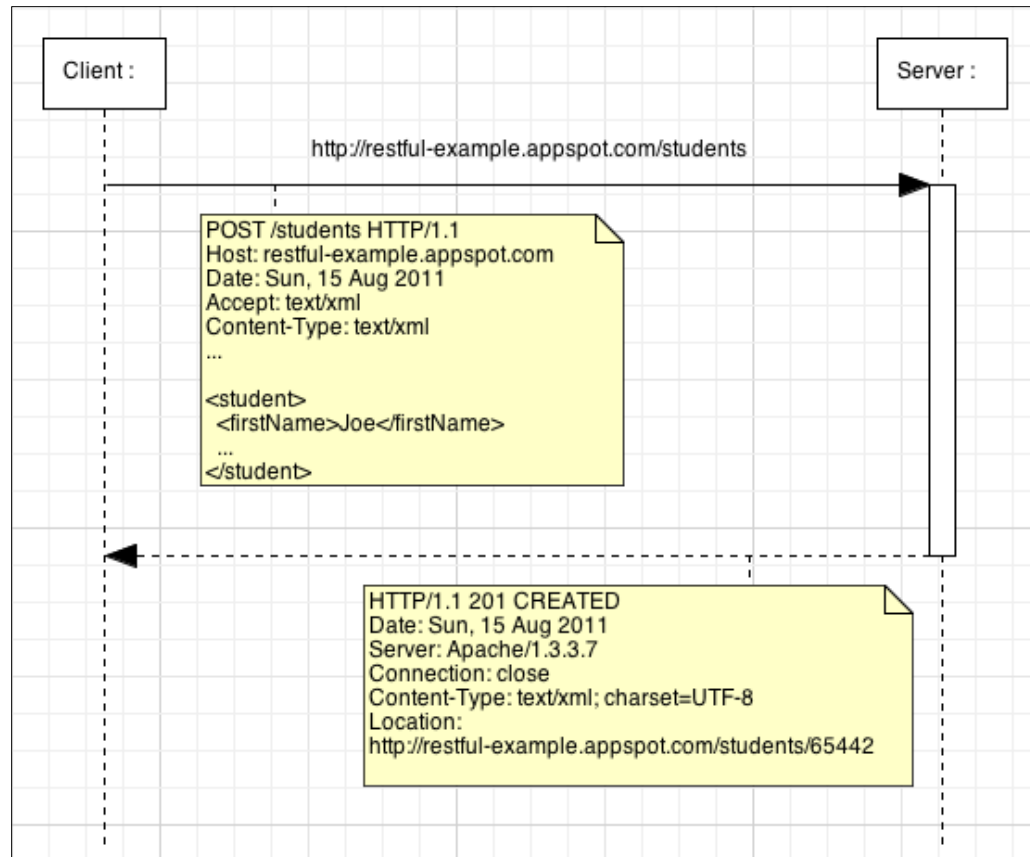
Par exemple, supposons que Joe n'existe pas dans notre liste et nous voulons l'ajouter:

Une représentation de Joe XML ressemble à ceci:

```
<student>  
  <birthDate>1946-06-28T22:00:13.524Z</birthDate>  
  <firstName>Joe</firstName>  
  <idBooster>65442</idBooster>  
  <lastName>Dalton</lastName>  
</student>
```

POST/ Create

Les détails de la requête



PUT/ Update

La méthode PUT est utilisée pour mettre à jour des ressources

Par exemple, supposons que la représentation actuelle de Joe est comme suit :

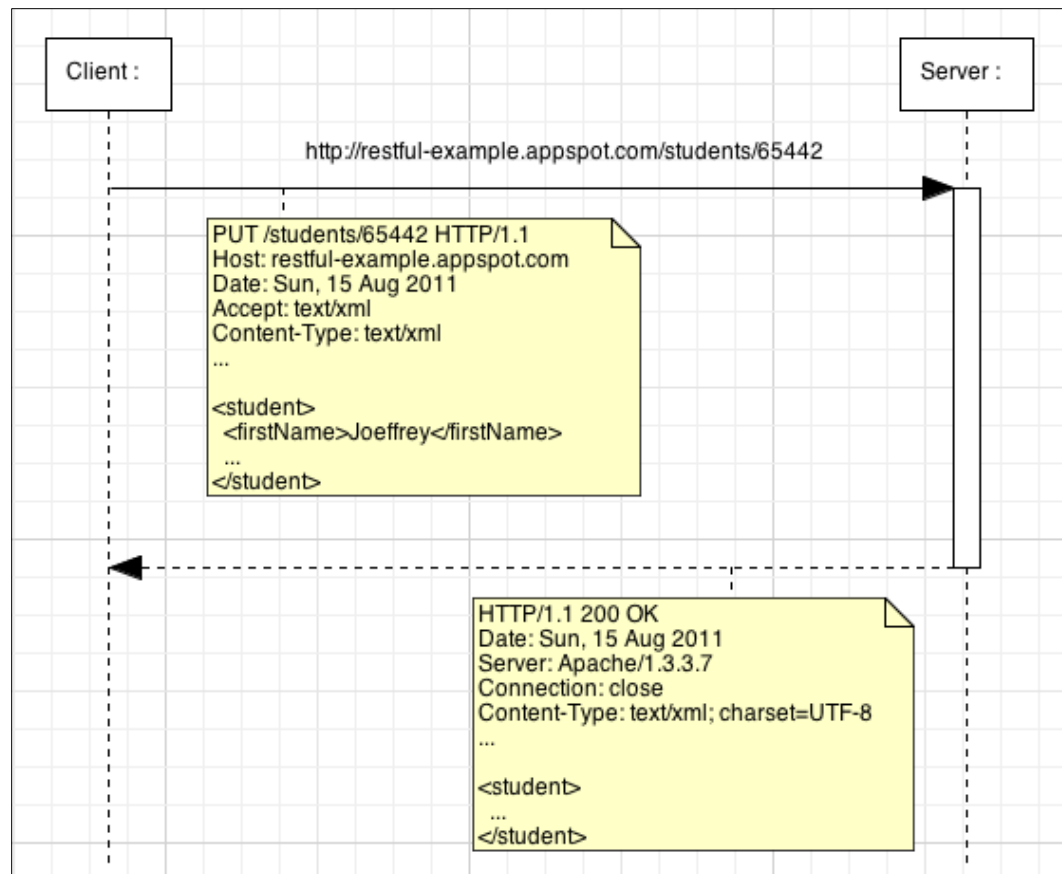
```
<student>  
  <birthDate>1946-06-28T22:00:13.524Z</birthDate>  
  <firstName>Joe</firstName>  
  <idBooster>65442</idBooster>  
  <lastName>Dalton</lastName>  
</student>
```

Nous voulons la mettre à jour pour être comme cela :

```
<student>  
  <birthDate>1946-06-28T22:00:13.524Z</birthDate>  
  <firstName>Joeffrey</firstName>  
  <idBooster>65442</idBooster>  
  <lastName>Dalton</lastName>  
</student>
```

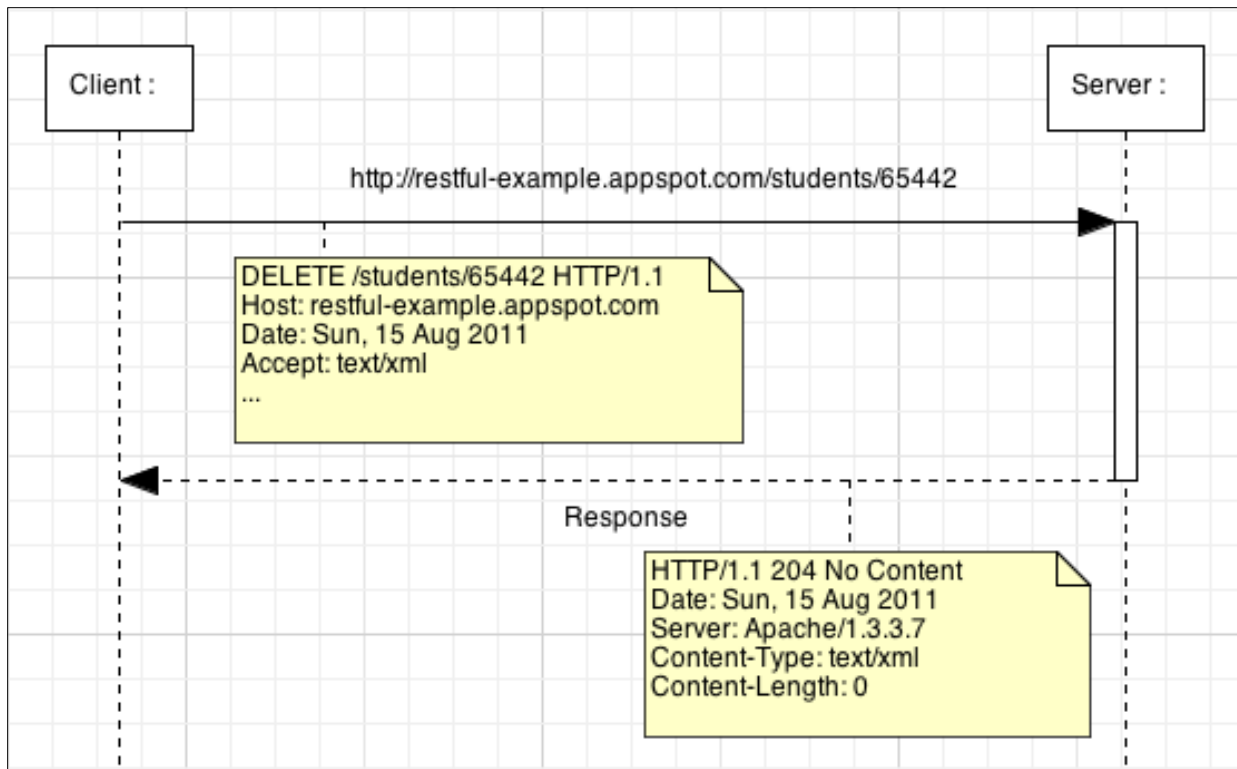
PUT / Update

Les détails de la requête



DELETE/ Delete

Les détails de la requête



Les services web REST avec JAX-RS

JAX-RS est l'une des spécifications Java EE

C'est une API dédiée à la conception de services web REST

Comme toutes les spécifications Java EE, plusieurs implémentations sont disponibles:

- **Apache CXF**, une implémentation open source
- **Jersey**, l'implémentation de référence de Sun / Oracle
- **RESTEasy**, l'implémentation de JBoss

Pour ce cours, nous utiliserons **Jersey**

Les services web REST avec JAX-RS

Ressource URI

Une ressource JAX-RS est une classe Java avec l'annotation **@Path** pour définir son URI.

Exemple :

```
@Path("/students")  
public class StudentsResource {  
    ...  
}
```

Cette annotation peut également être utilisée pour définir des variables dans des URI

```
@Path("/students/{idBooster}")  
public class StudentsResource {  
    ...  
}
```

Les services web REST avec JAX-RS

Méthodes Http

Pour déterminer quelle méthode Java appeler en fonction de la demande de méthode HTTP, JAX-RS fournit quatre annotations:

@GET

@POST

@PUT

@DELETE

Exemple :

```
@Path("/students")
public class StudentsResource {
    @GET
    public String handleGetRequest() { ... }

    @POST
    public String handlePostRequest(String payload) { ... }
}
```

Les services web REST avec JAX-RS

Chemins relatifs dans les méthodes

Dans certains cas, il peut être utile de définir deux méthodes Java associées à deux adresses URI différentes dans une même classe.

On peut faire cela en déclarant l'annotation **@Path** sur une méthode

Exemple :

```
@Path("/students")
public class StudentsResource {

    @GET
    public String getStudentList() { ... }

    @GET @Path("/search/{query}")
    public String searchStudent() { ... }
}
```

Dans ce cas, une requête GET sur **/students/search/Jack** exécutera la méthode **searchStudent()**

Les services web REST avec JAX-RS

URI variables

Nous avons vu auparavant comment définir des variables avec l'annotation `@Path`, mais pas comment les récupérer dans une méthode Java

Pour ce faire, JAX-RS définit une annotation **`@PathParam`**

On peut l'utiliser comme suit:

```
@GET @Path("/{idBooster}")
public String getStudent(@PathParam("idBooster") Long id) {
    ...
}
@GET @Path("/search/{query}")
public String searchStudent(@PathParam("query") String q) {
    ...
}
```

Les services web REST avec JAX-RS

Formats de l'entrée

L'annotation **@Consumes** permet de connaître le format des données de l'entrée.

Exemple :

```
@Path("/students")
public class StudentResource {

    @POST @Consumes(MediaType.APPLICATION_XML)
    public String addXmlStudent(String payload) { ... }

    @POST @Consumes(MediaType.APPLICATION_JSON)
    public String addJsonStudent(String payload) { ... }

}
```

Les services web REST avec JAX-RS

Formats de la sortie

L'annotation **@Produces** permet de connaître le format des données de la sortie.

Exemple :

```
@Path("/students")
public class StudentResource {

    @GET @Produces(MediaType.APPLICATION_XML)
    public String getStudentAsXml() { ... }

    @GET @Produces(MediaType.APPLICATION_JSON)
    public String getStudentAsJson() { ... }

}
```

JAX-RS avec JAXB

Présentation

JAXB est une API qui permet de mapper (faire le lien entre) les classes Java à la représentation XML.

Cela est représenté principalement en deux actions :

- Marshal Java objects into XML
- Unmarshal XML into Java objects

Très utile pour les services Web !

Cela est assuré par les annotations :

@XmlRootElement

@XmlElement

@XmlAttribute

JAX-RS avec JAXB

XMLRootElement

Mappe une classe à un élément XML.

Par défaut, le nom de l'élément XML est identique à celui de la classe.

Mais on peut le changer grâce à l'attribut **name** de l'annotation

```
@XmlRootElement(name="doc")  
public class Document {  
  
}
```

JAX-RS avec JAXB

XmlElement

Mappe une propriété à un élément XML

Cette annotation est optionnelle

Trois attributs d'annotation sont disponibles:

name: le nom de l'élément XML

required: spécifie si l'élément est optionnel ou pas.

defaultValue: valeur par défaut de cet élément

```
@XmlElement(name = "first-name", required = true)  
private String firstName;
```

JAX-RS avec JAXB

XMLAttribut

Mappe une propriété à un attribut XML

Deux attributs d'annotation sont disponibles:

name: le nom de l'élément XML

required: spécifie si l'élément est optionnel ou non.

```
@XmlAttribute  
private int score;
```

JAX-RS avec JAXB

Exemple

```
@XmlRootElement
public class User {
    private String username;
    private String firstName;
    private String lastName;
    // Getters and Setters
}
```



```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<xs:schema version="1.0" xmlns:xs="...">
  <xs:element name="user" type="user"/>
  <xs:complexType name="user">
    <xs:sequence>
      <xs:element name="firstName" type="xs:string" minOccurs="0"/>
      <xs:element name="lastName" type="xs:string" minOccurs="0"/>
      <xs:element name="username" type="xs:string" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

JAX-RS avec JAXB

Exemple

```
@XmlRootElement
public class User {

    // Attributes

    @XmlAttribute(name="id-booster")
    public Long getIdBooster() { return idBooster; }

    @XmlElement(name="first-name")
    public String getFirstName() { return firstName; }

    @XmlElement(name="last-name")
    public String getLastName() { return lastName; }

    // Setters

}
```

JAX-RS avec JAXB

Exemple

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<xs:schema version="1.0" xmlns:xs="...">
  <xs:element name="user" type="user"/>
  <xs:complexType name="user">
    <xs:sequence>
      <xs:element name="first-name" type="xs:string"
minOccurs="0"/>
      <xs:element name="last-name" type="xs:string"
minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="id-booster" type="xs:long"/>
  </xs:complexType>
</xs:schema>
```

TP 5 - REST

Travail à réaliser (1/2)

- 1) Créer un projet MAVEN (en utilisant l'Archtype "heroku stuff")
- 2) Dans le fichier POM.xml dé-commenter la dépendance JSON

```
<dependency>
```

```
  <groupId>org.glassfish.jersey.media</groupId>
```

```
  <artifactId>jersey-media-json-binding</artifactId>
```

```
</dependency>
```

- 3) Créer une classe "Compte" (dans le package "metier")

Attributs : int code / double solde / Date dateCreation (+ Getters & Setters + Constructeurs)

- 4) Créer une classe "BanqueRESTService" (dans le package "rest")

Créer 3 méthodes : double conversion(double mt) / Compte consulterCompte(int code) / List<Compte> listComptes()

TP 5 - REST

Travail à réaliser (2/2)

Ajouter les annotations nécessaires pour transformer ce POJO en service web REST (au format JSON)

5) Exécuter le projet puis lancer le web service créé avec les liens suivants :

<http://localhost:8080/TP3-RESTWS/banque/conversion/2>

<http://localhost:8080/TP3-RESTWS/banque/comptes/1>

<http://localhost:8080/TP3-RESTWS/banque/comptes>

6) Ajouter la possibilité de produire les données sous format XML pour les 3 services

7) Installer le plugin "Advanced REST Client" dans votre navigateur afin de faciliter la visualisation et l'utilisation des WS REST créés.